

**UNIVERSIDADE TECNOLÓGICA FEDERAL DO
PARANÁ
CURSO DE ENGENHARIA DE COMPUTAÇÃO**

ELEQ30 - Análise de Sistemas Lineares
Professor César Janeczko

Relatório: Prática 5

Filtros IIR (Filtros Recursivos) - Som

Disciplina: Análise de Sistemas Lineares

Leonardo Pereira Shibata

Sergio Roncato Maccari

Curitiba, Junho de 2026

1 Filtros de pólo simples (R-C): passa-baixas e passa-altas

Enunciado: “Implemente rotina MATLAB para calcular coeficientes de filtros de Pólo simples (R-C) passa-baixas e passa-altas. Você escolhe f_c . Encontre os gráficos da resposta ao impulso (1 0 0 0 0 0....(100 pontos)) e das respostas em frequência (ganho, atenuação e fase).”

O filtro passa-baixas de pólo simples é o equivalente discreto de um circuito R-C. A partir da frequência de corte normalizada f_c define-se a constante $r = e^{-2\pi f_c}$, que posiciona o único pólo do sistema. Quanto mais próximo de 1 for r , mais lenta é a resposta ao impulso e menor é a frequência de corte. A equação de diferenças é $y[n] = a_0 x[n] + b_1 y[n-1]$, com $a_0 = 1 - r$ e $b_1 = r$, garantindo ganho unitário em corrente contínua. Escolheu-se $f_c = 0,10$.

Listing 1: Filtro passa-baixas IIR de pólo simples.

```
1 function y = CalculaFPBIIR(fc, x)
2 % CALCULAFPIIR Filtro passa-baixas IIR de polo simples (analogo a um R-C).
3 % y = CalculaFPBIIR(fc, x) filtra o vetor x com um polo simples cuja
4 % frequencia de corte normalizada e fc (0 < fc < 0.5).
5 %
6 % Coeficientes: r = exp(-2*pi*fc); a0 = 1 - r; b1 = r;
7 % Equacao de diferencas: y[n] = a0*x[n] + b1*y[n-1]
8     nx = length(x);
9     r = exp(-2*pi*fc);
10    a0 = 1 - r;
11    b1 = r;
12    y = zeros(1, nx);
13    y(1) = a0*x(1);
14    for i = 2:nx
15        y(i) = a0*x(i) + b1*y(i-1);
16    end
17 end
```

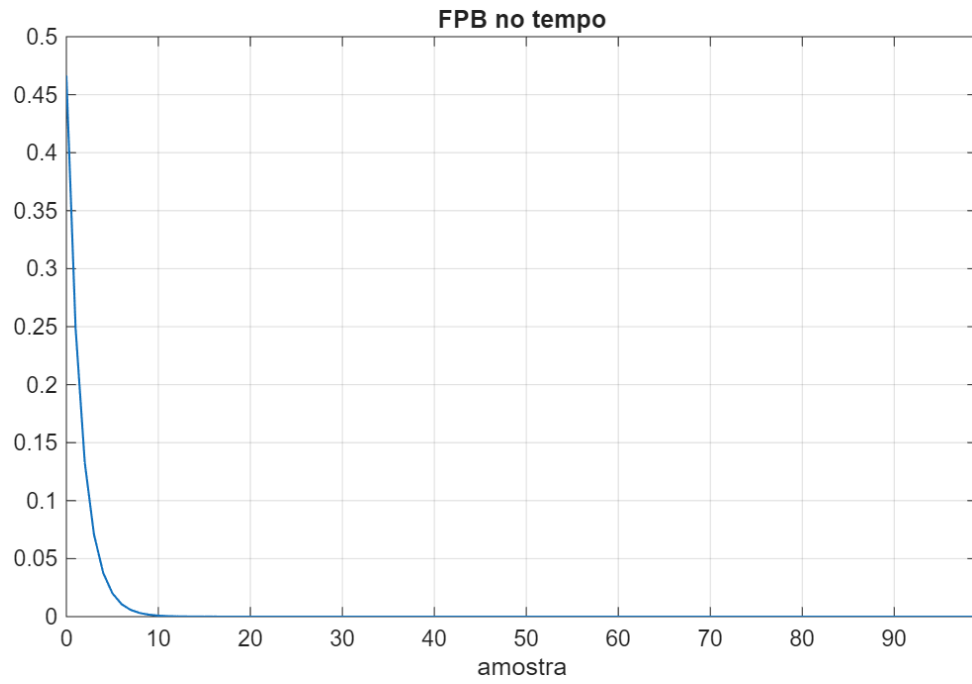


Figura 1: Resposta ao impulso do FPB de pólo simples no tempo ($f_c = 0,10$), calculada sobre 100 pontos.

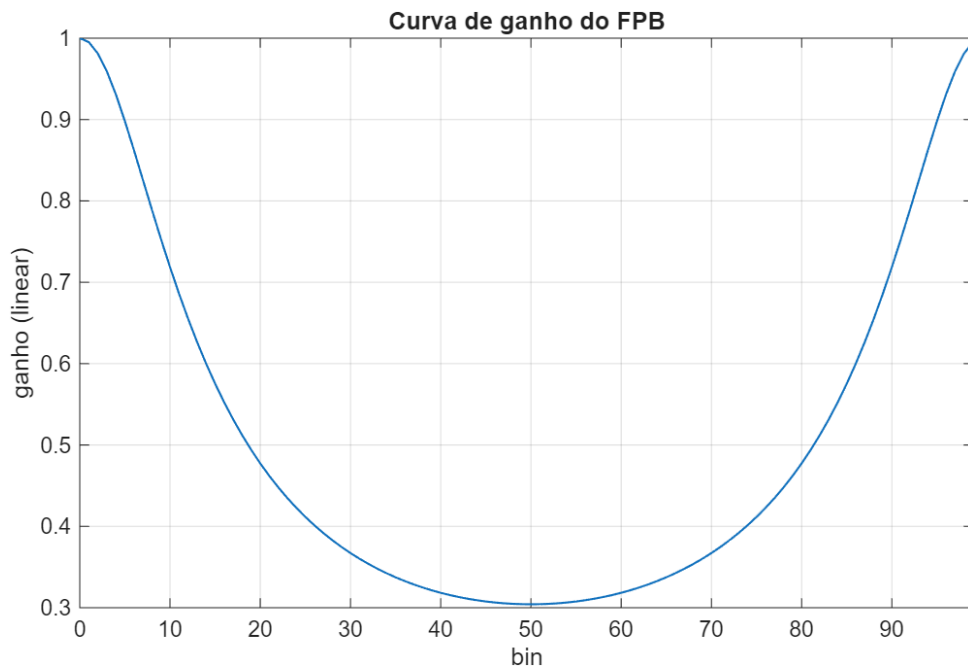


Figura 2: Curva de ganho (linear) do FPB de pólo simples ($f_c = 0,10$).

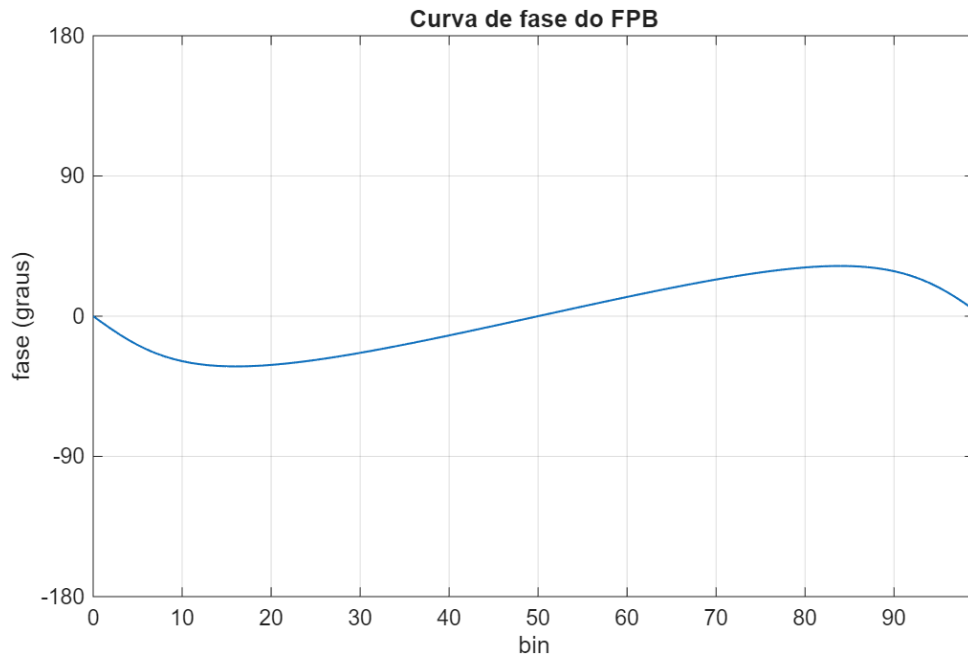


Figura 3: Curva de fase (em graus) do FPB de pólo simples ($f_c = 0,10$).

A resposta ao impulso decai exponencialmente, como esperado de um sistema de primeira ordem. A curva de ganho parte de 1 em corrente contínua e cai suavemente até a frequência de Nyquist (índice 50), evidenciando a transição gradual típica de um único pólo. A simetria em torno do índice 50 decorre da FFT de um sinal real.

O passa-altas é obtido pela versão complementar do mesmo pólo: mantém-se $b_1 = r$ e usam-se $a_0 = (1 + r)/2$ e $a_1 = -(1 + r)/2$, de modo que a componente contínua é cancelada e as altas frequências são preservadas.

Listing 2: Filtro passa-altas IIR de pólo simples.

```

1 function y = CalculaFPAlIIR(fc, x)
2 % CALCULAFPAIIR Filtro passa-altas IIR de polo simples (analogo a um R-C).
3 % y = CalculaFPAlIIR(fc, x) filtra o vetor x com a versao passa-altas do
4 % polo simples, com frequencia de corte normalizada fc (0 < fc < 0.5).
5 %
6 % Coeficientes: r = exp(-2*pi*fc); a0 = (1+r)/2; a1 = -(1+r)/2; b1 = r;
7 % Equacao de diferencas: y[n] = a0*x[n] + a1*x[n-1] + b1*y[n-1]
8     nx = length(x);
9     r = exp(-2*pi*fc);
10    a0 = (1 + r)/2;
11    a1 = -(1 + r)/2;
12    b1 = r;
13    y = zeros(1, nx);
14    y(1) = a0*x(1);
15    for i = 2:nx
16        y(i) = a0*x(i) + a1*x(i-1) + b1*y(i-1);
17    end
18 end

```

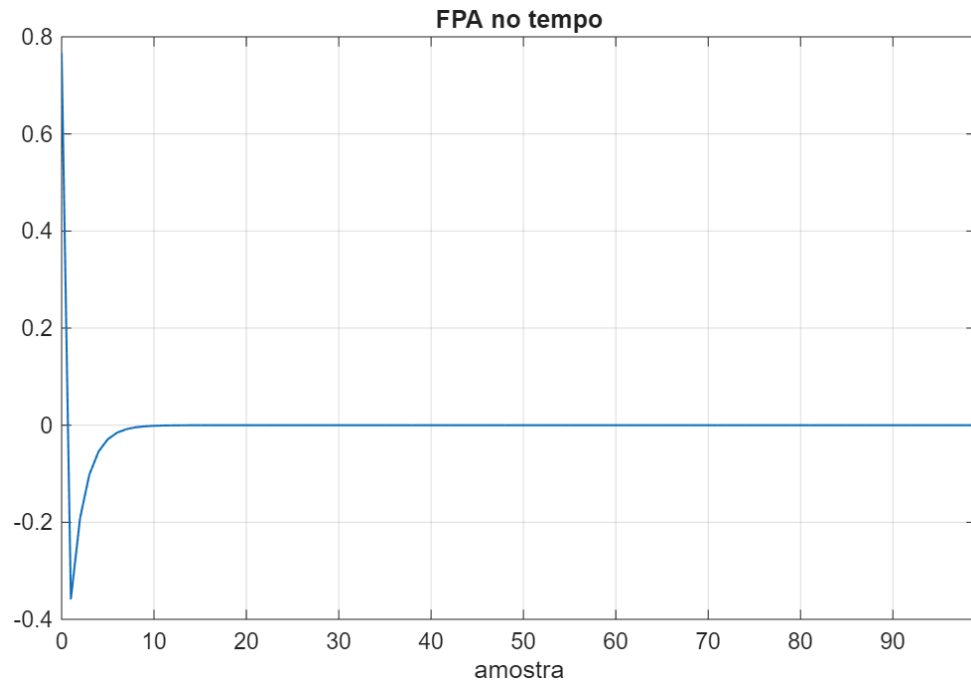


Figura 4: Resposta ao impulso do FPA de pólo simples no tempo ($f_c = 0,10$).

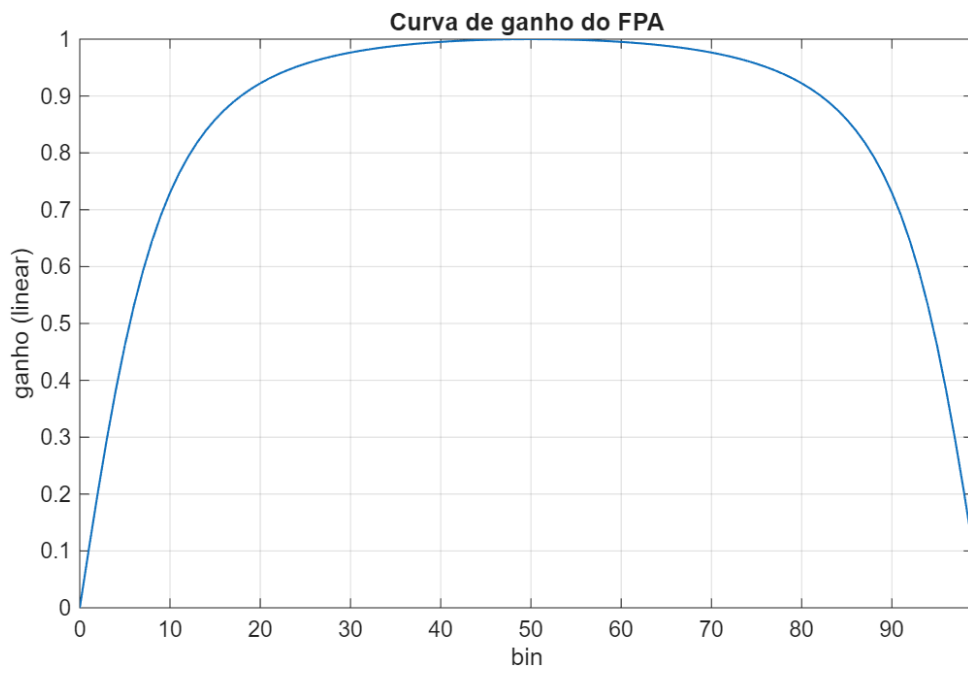


Figura 5: Curva de ganho (linear) do FPA de pólo simples ($f_c = 0,10$).

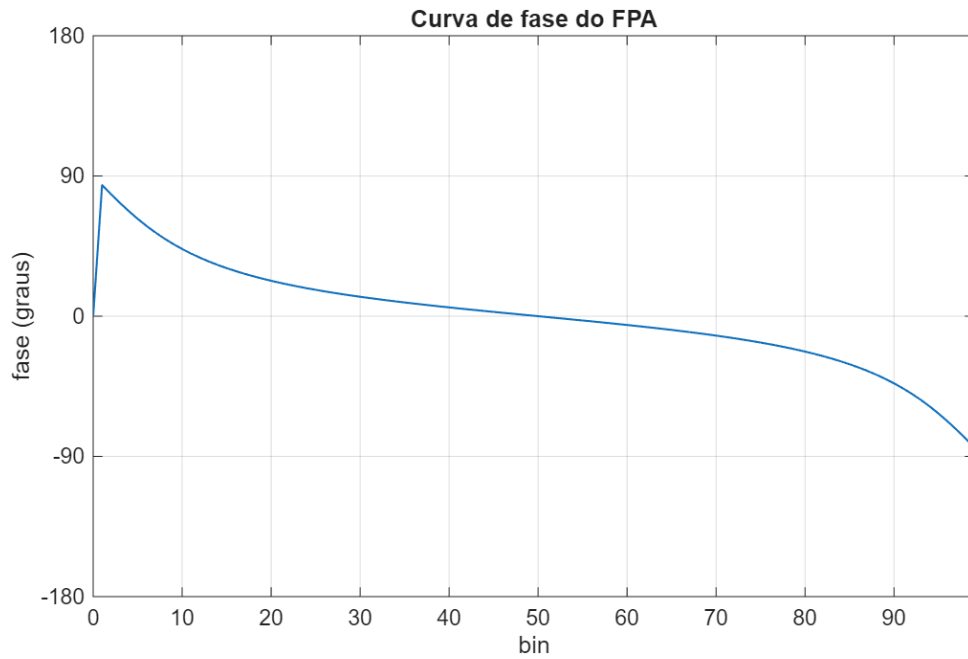


Figura 6: Curva de fase (em graus) do FPA de pólo simples ($f_c = 0,10$).

No FPA o ganho é praticamente nulo em corrente contínua e cresce em direção às altas frequências, comportamento oposto ao do FPB. Por ser de primeira ordem, a transição também é suave.

2 Teste dos filtros FPB e FPA

Enunciado: “Elabore um sinal com baixa frequência + alta frequência e teste os filtros FPB e FPA calculados com as rotinas anteriores do item 1). Mostre o sinal gerado e os sinais filtrados (no tempo e na frequência).”

Gerou-se um sinal com 1000 amostras ($F_s = 1000$ Hz) somando uma componente de baixa frequência (30 Hz) e uma de alta frequência (350 Hz): $x[n] = \sin(2\pi 30 n/1000) + \sin(2\pi 350 n/1000)$.

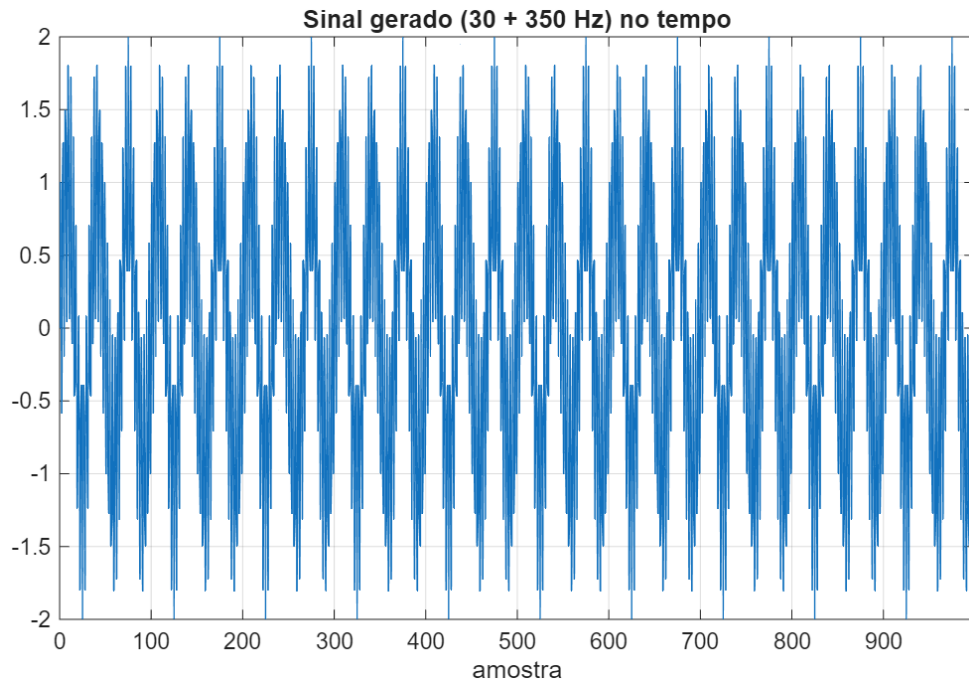


Figura 7: Sinal gerado (30 Hz + 350 Hz) no tempo.

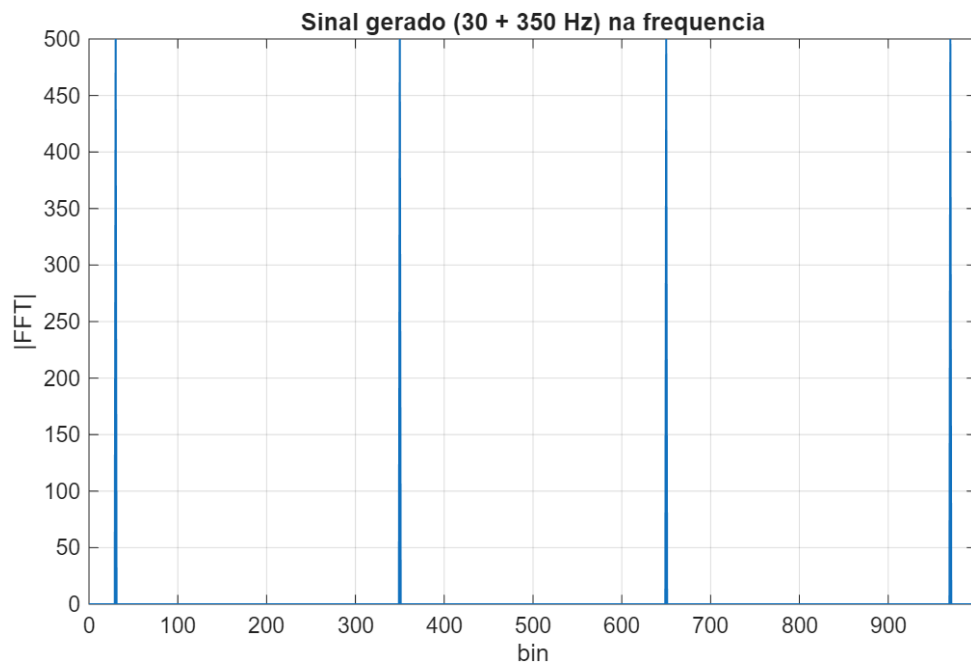


Figura 8: Sinal gerado (30 Hz + 350 Hz) na frequência. As raias aparecem em 30 e 350 Hz, com os espelhos em torno dos índices 650 e 970 (espelhamento da FFT em torno de $F_s/2$).

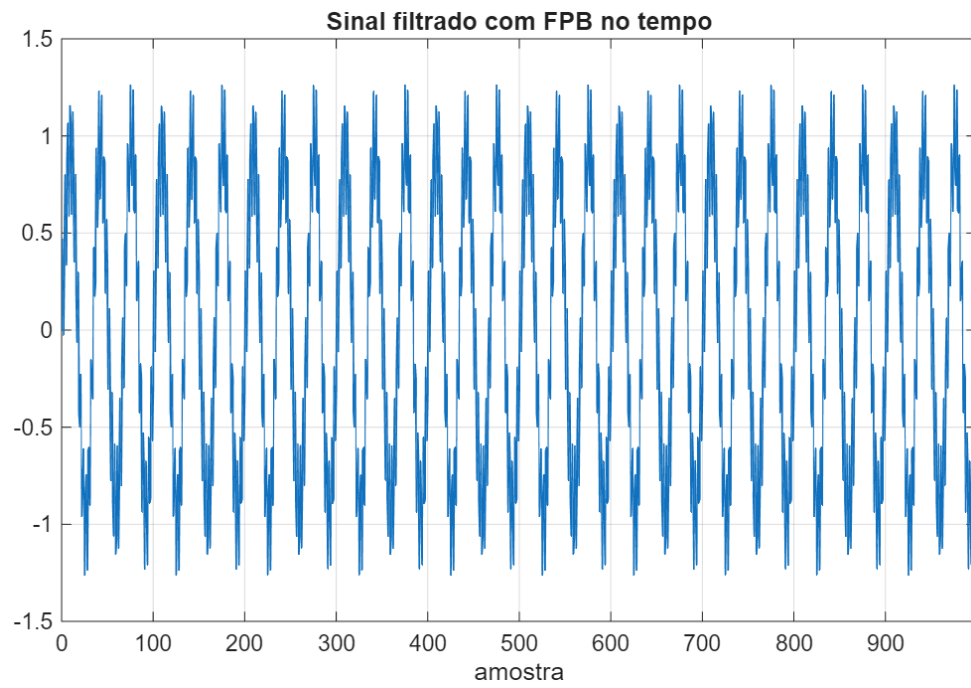


Figura 9: Sinal filtrado com o FPB no tempo.

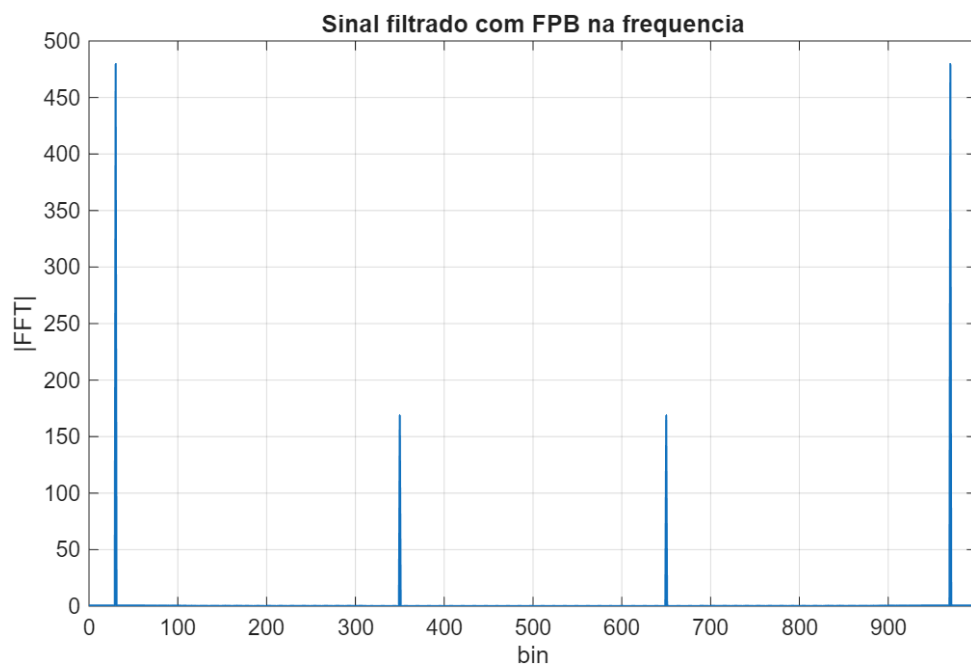


Figura 10: Sinal filtrado com o FPB na frequência: a raia de 30 Hz é preservada e a de 350 Hz é atenuada.

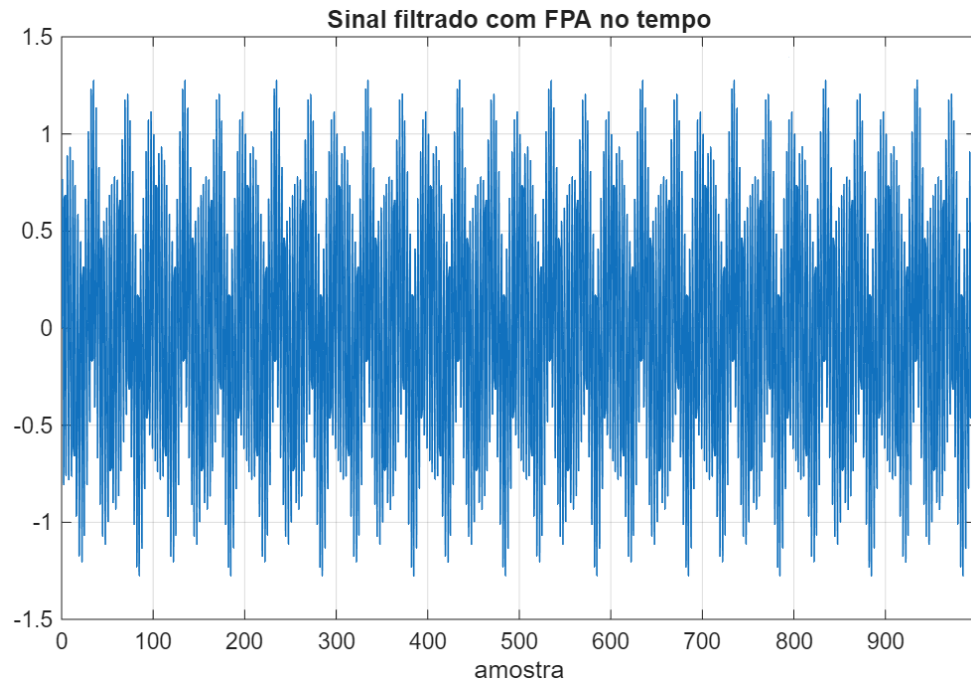


Figura 11: Sinal filtrado com o FPA no tempo.

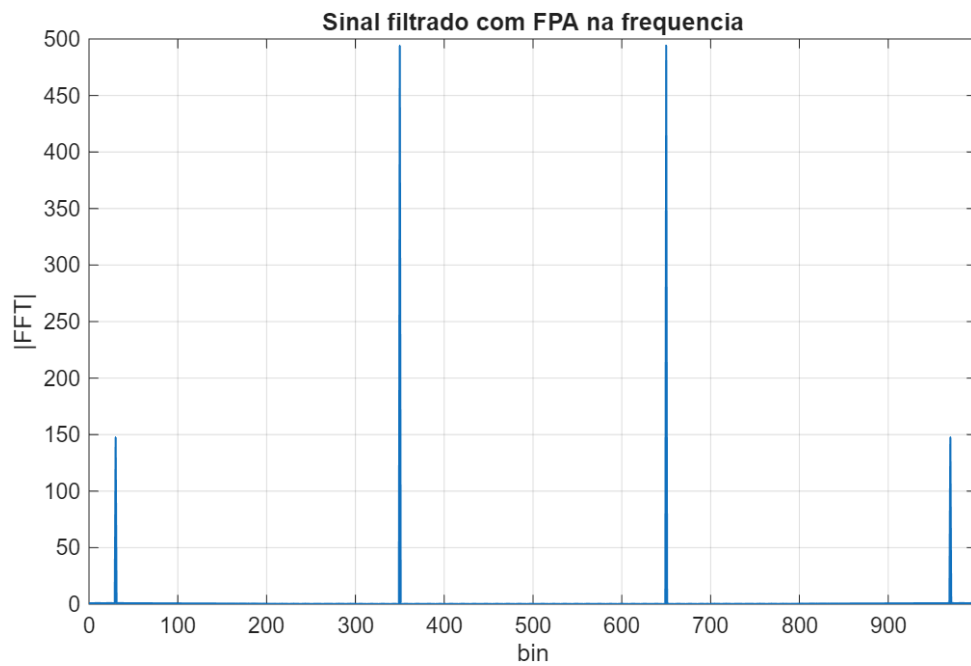


Figura 12: Sinal filtrado com o FPA na frequência: a raia de 350 Hz é preservada e a de 30 Hz é atenuada.

No espectro do sinal gerado aparecem as raiais em 30 e 350 Hz (e seus espelhos). Após o FPB, a raia de 30 Hz é preservada e a de 350 Hz é fortemente atenuada; após o FPA ocorre o inverso. Como os filtros são de pólo simples, a atenuação da componente indesejada é parcial (não há corte abrupto), o que é coerente com as curvas de ganho do item 1.

3 Filtros de banda estreita: passa-banda e rejeita-banda

Enunciado: “Implemente rotina MATLAB para calcular coeficientes de filtros Banda estreita (passa-banda e corta-banda). Você escolhe f_c e BW . OBS: BW dever ser estreito! Encontre os gráficos da resposta ao impulso (1 0 0 0 0 0....(100 pontos)) e das respostas em frequência (ganho, atenuação e fase).”

Os filtros de banda estreita são sistemas recursivos de segunda ordem (resonadores). A largura de banda BW define a posição dos pólos por meio de $r = 1 - 3BW$ (quanto menor o BW , mais próximo de 1 fica r e mais estreita é a banda). O fator de normalização é

$$K = \frac{1 - 2r \cos(2\pi f) + r^2}{2 - 2 \cos(2\pi f)}.$$

Escolheu-se a frequência central $f_c = 0,25$ e $BW = 0,008$ (banda estreita).

Listing 3: Filtro recursivo passa-banda de banda estreita.

```
1 function y = FiltroPassaBanda(bw, f, x)
2 % FILTROPASSABANDA Filtro recursivo passa-banda de banda estreita.
3 % y = FiltroPassaBanda(bw, f, x) realca uma faixa estreita de largura bw
4 % centrada na frequencia normalizada f (0 < f < 0.5).
5 % (Projeto recursivo de 2a ordem - Smith, The Eng. Guide to DSP, cap. 19.)
6 nx = length(x);
7 r = 1 - 3*bw;
8 k = (1 - 2*r*cos(2*pi*f) + r^2) / (2 - 2*cos(2*pi*f));
9 a0 = 1 - k;
10 a1 = 2*(k - r)*cos(2*pi*f);
11 a2 = r^2 - k;
12 b1 = 2*r*cos(2*pi*f);
13 b2 = -r^2;
14 y = zeros(1, nx);
15 y(1) = a0*x(1);
16 y(2) = a0*x(2) + a1*x(1) + b1*y(1);
17 for i = 3:nx
18     y(i) = a0*x(i) + a1*x(i-1) + a2*x(i-2) + b1*y(i-1) + b2*y(i-2);
19 end
20 end
```

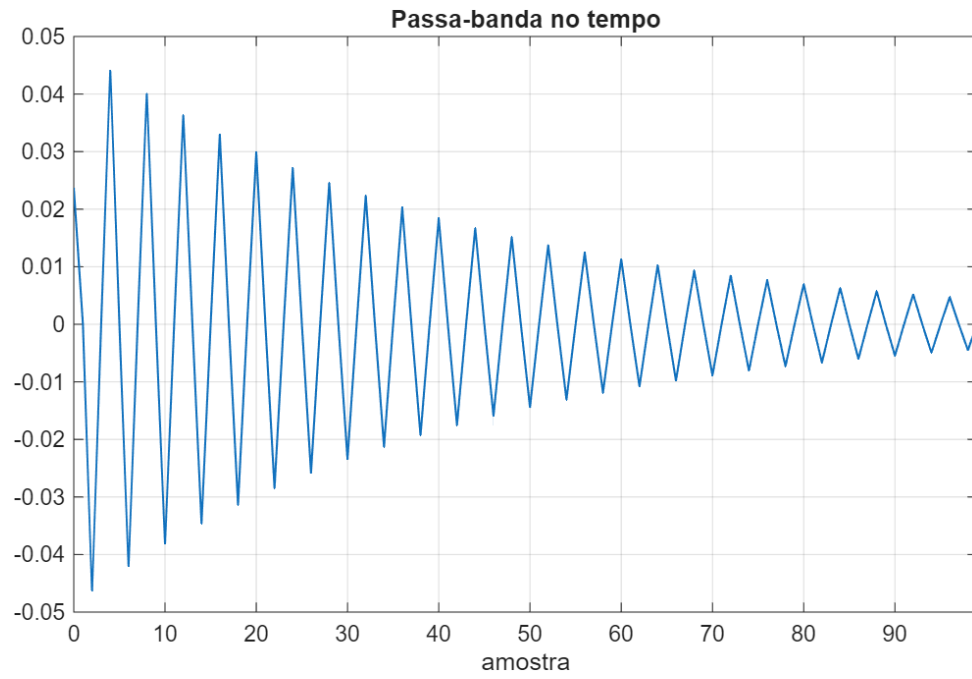


Figura 13: Resposta ao impulso do passa-banda no tempo (senoide amortecida) ($f_c = 0,25$, $BW = 0,008$).

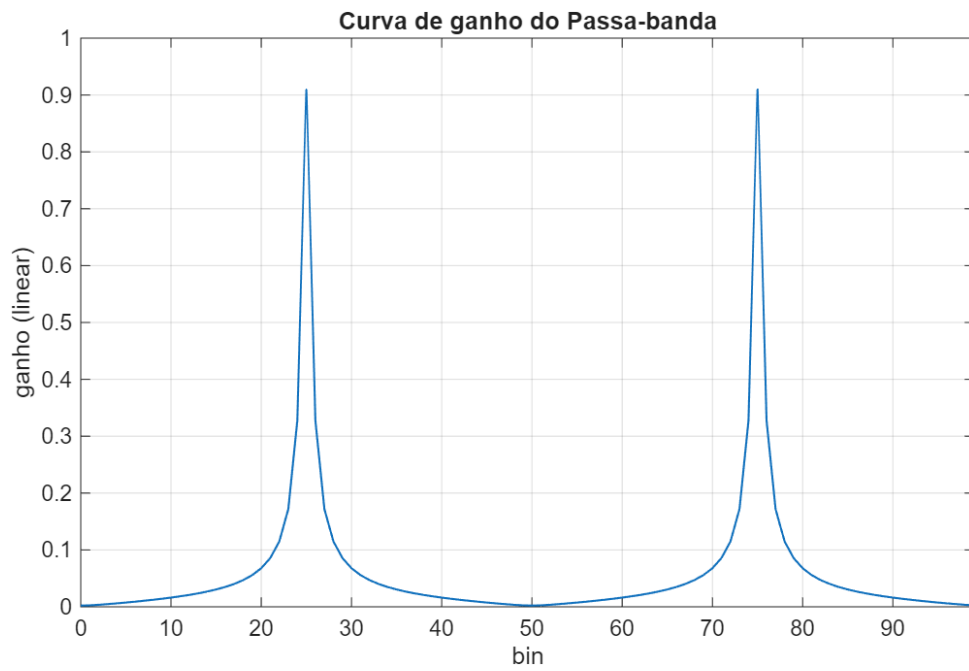


Figura 14: Curva de ganho (linear) do passa-banda ($f_c = 0,25$, $BW = 0,008$).

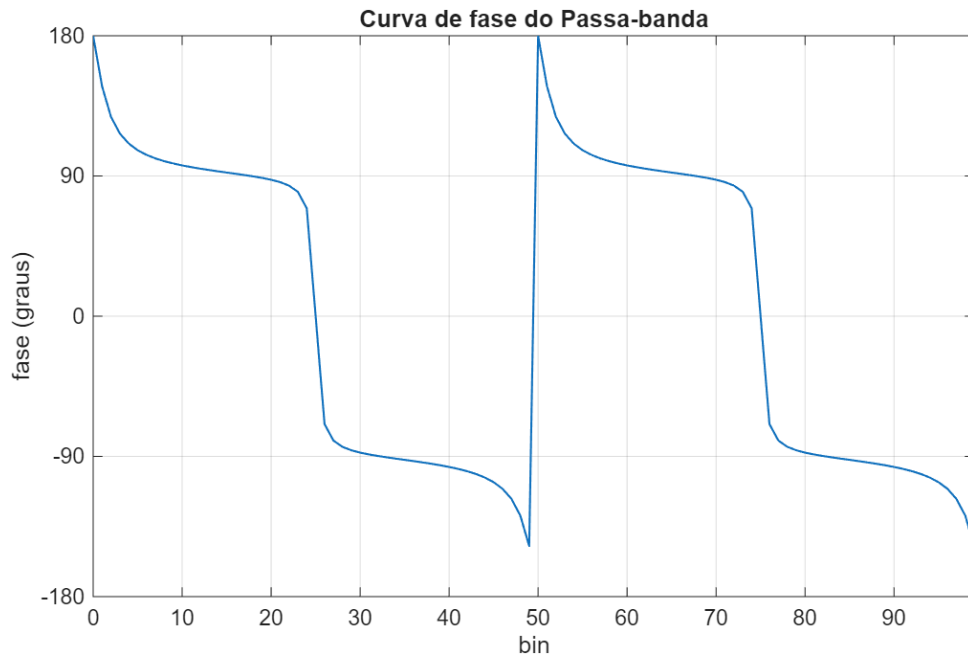


Figura 15: Curva de fase (em graus) do passa-banda ($f_c = 0,25$, $BW = 0,008$).

Listing 4: Filtro recursivo rejeita-banda (notch) de banda estreita.

```

1 function y = FiltroRejeitaBanda(bw, f, x)
2 % FILTROREJEITABANDA Filtro recursivo rejeita-banda (notch) de banda estreita.
3 % y = FiltroRejeitaBanda(bw, f, x) atenua uma faixa estreita de largura bw
4 % centrada na frequencia normalizada f (0 < f < 0.5).
5 % (Projeto recursivo de 2a ordem - Smith, The Eng. Guide to DSP, cap. 19.)
6     nx = length(x);
7     r = 1 - 3*bw;
8     k = (1 - 2*r*cos(2*pi*f) + r^2) / (2 - 2*cos(2*pi*f));
9     a0 = k;
10    a1 = -2*k*cos(2*pi*f);
11    a2 = k;
12    b1 = 2*r*cos(2*pi*f);
13    b2 = -r^2;
14    y = zeros(1, nx);
15    y(1) = a0*x(1);
16    y(2) = a0*x(2) + a1*x(1) + b1*y(1);
17    for i = 3:nx
18        y(i) = a0*x(i) + a1*x(i-1) + a2*x(i-2) + b1*y(i-1) + b2*y(i-2);
19    end
20 end

```

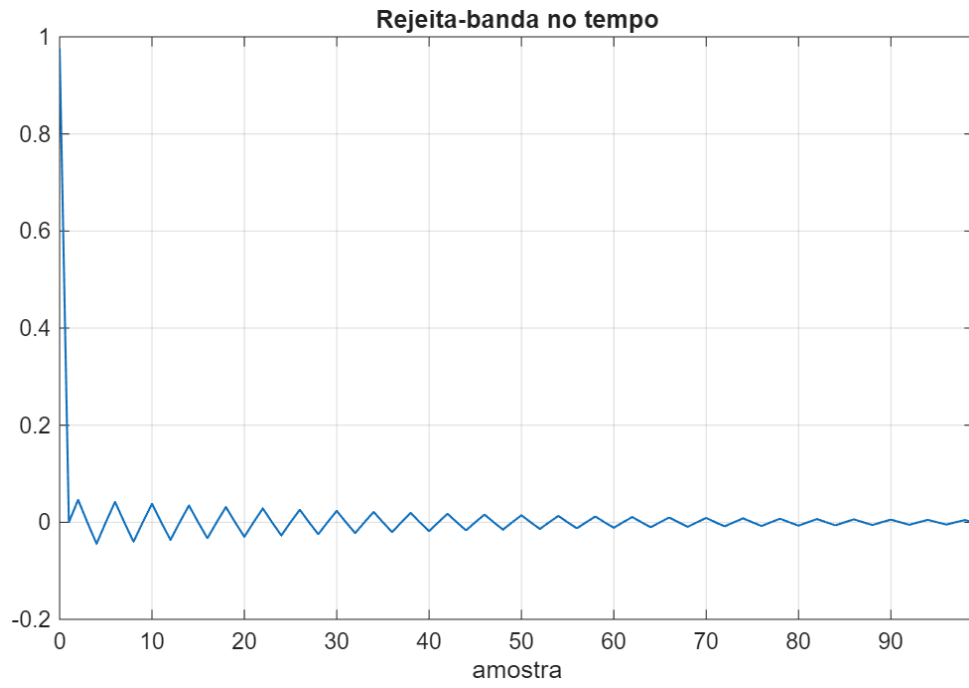


Figura 16: Resposta ao impulso do rejeita-banda no tempo ($f_c = 0,25$, $BW = 0,008$).

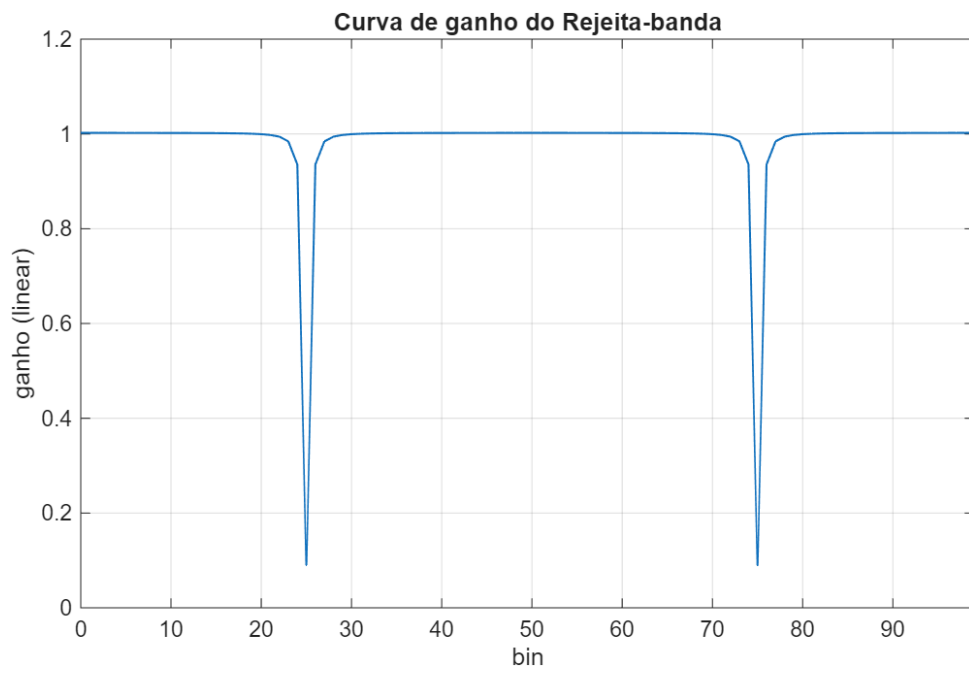


Figura 17: Curva de ganho (linear) do rejeita-banda ($f_c = 0,25$, $BW = 0,008$).

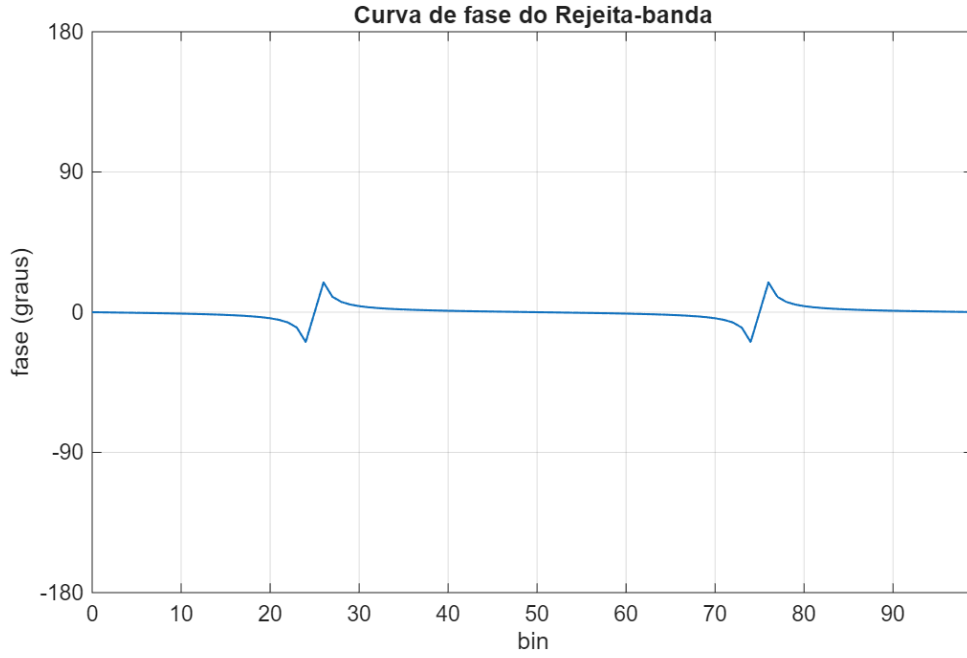


Figura 18: Curva de fase (em graus) do rejeita-banda ($f_c = 0,25$, $BW = 0,008$).

A resposta ao impulso do passa-banda é uma senoide amortecida, cuja taxa de decaimento depende de r . No domínio da frequência, o passa-banda apresenta um pico estreito no índice 25 (e o espelho no índice 75), enquanto o rejeita-banda apresenta um entalhe profundo exatamente nessas posições, mantendo ganho unitário no restante do espectro.

4 Teste dos filtros de banda estreita

Enunciado: “Elabore um sinal com várias frequências a sua escolha, sinal este que seja compatível com as frequências de corte dos filtros implementados no item 3). Teste os filtros passa-banda e corta-banda, banda estreita, calculados com a rotina anterior, item 3). Mostre o sinal gerado e os sinais filtrados (no tempo e na frequência).”

Gerou-se um sinal com cinco senoides ($F_s = 1000$ Hz): 40, 120, 250, 380 e 470 Hz. A componente de 250 Hz coincide com a frequência central dos filtros ($f_c = 0,25 \Rightarrow 250$ Hz), permitindo demonstrar o isolamento e a rejeição seletivos.

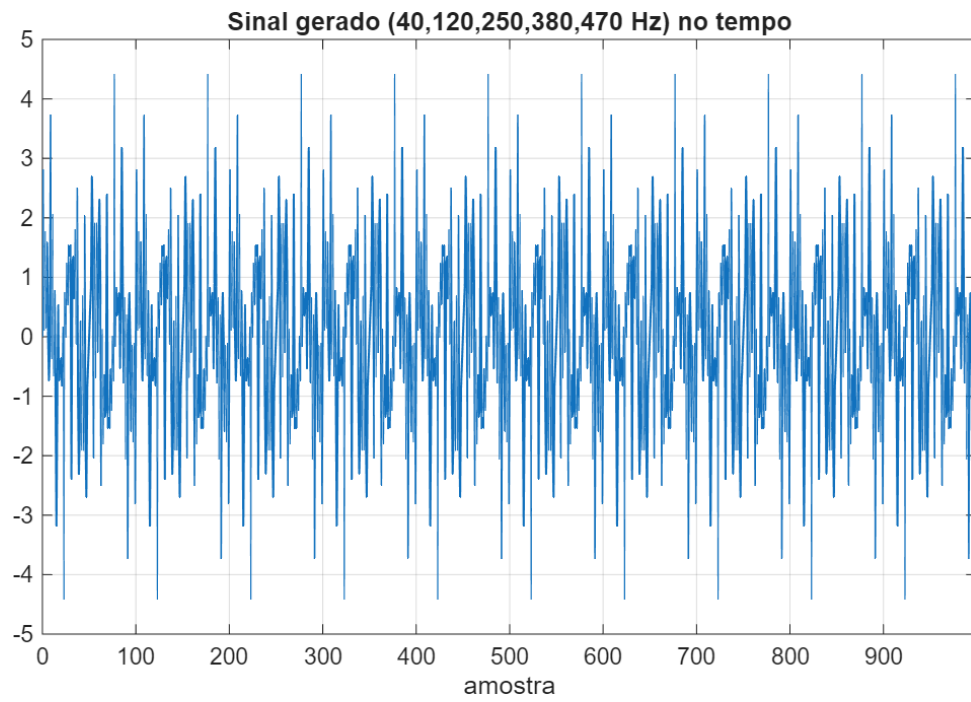


Figura 19: Sinal gerado (40, 120, 250, 380 e 470 Hz) no tempo.

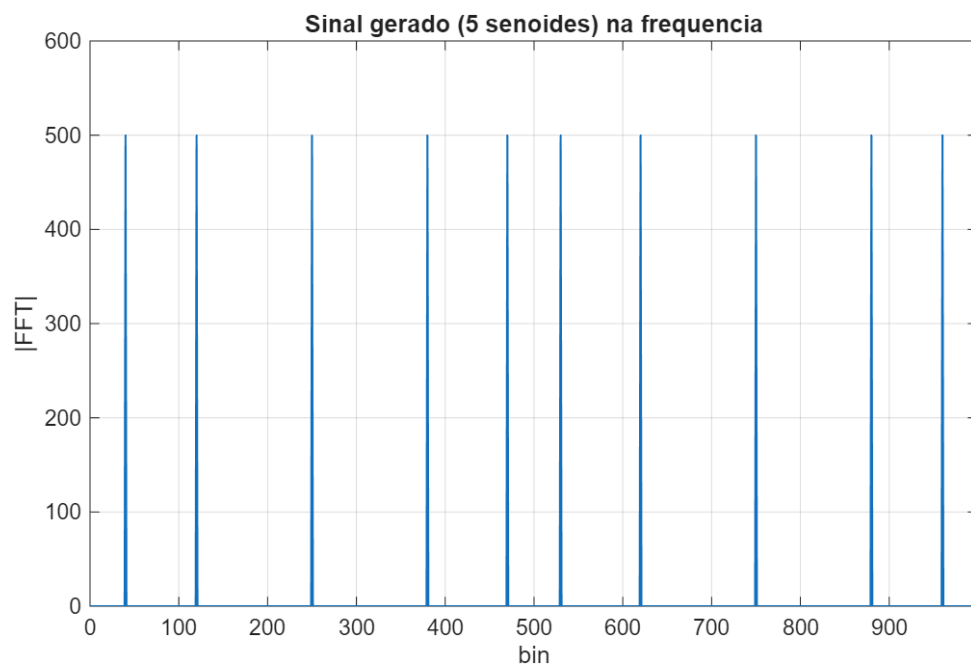


Figura 20: Sinal gerado (cinco senoides) na frequência: raias em 40, 120, 250, 380 e 470 Hz (e seus espelhos).

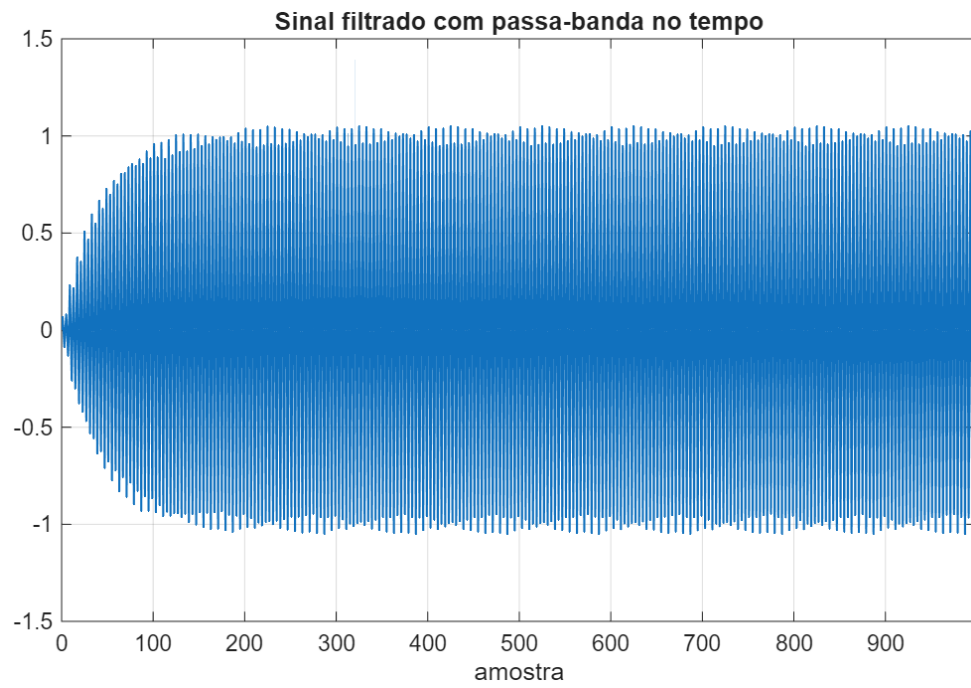


Figura 21: Sinal filtrado com o passa-banda (centrado em 250 Hz) no tempo.

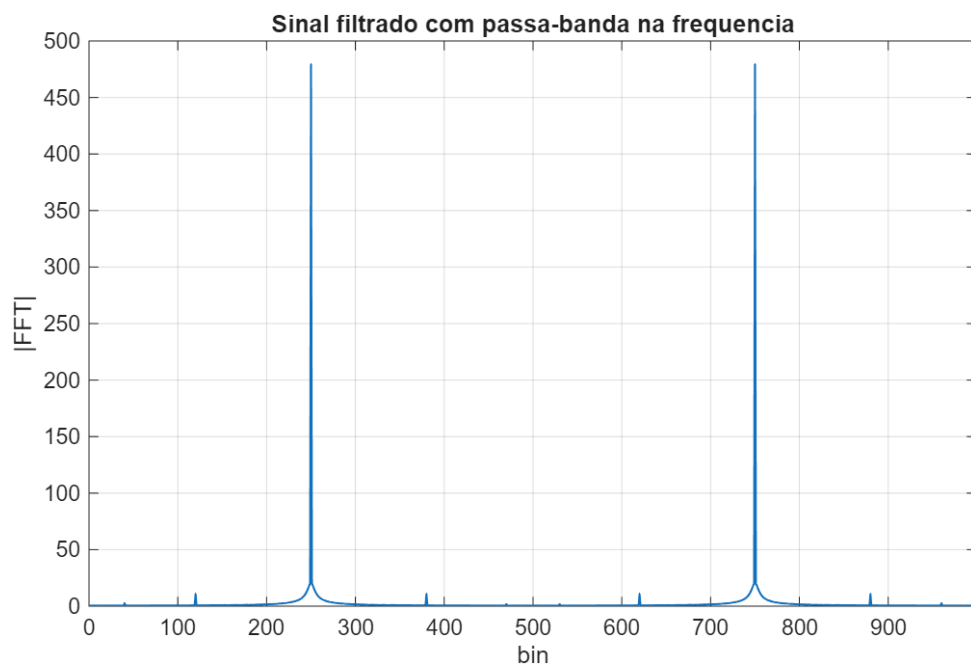


Figura 22: Sinal filtrado com o passa-banda na frequência: isola praticamente apenas a componente de 250 Hz.

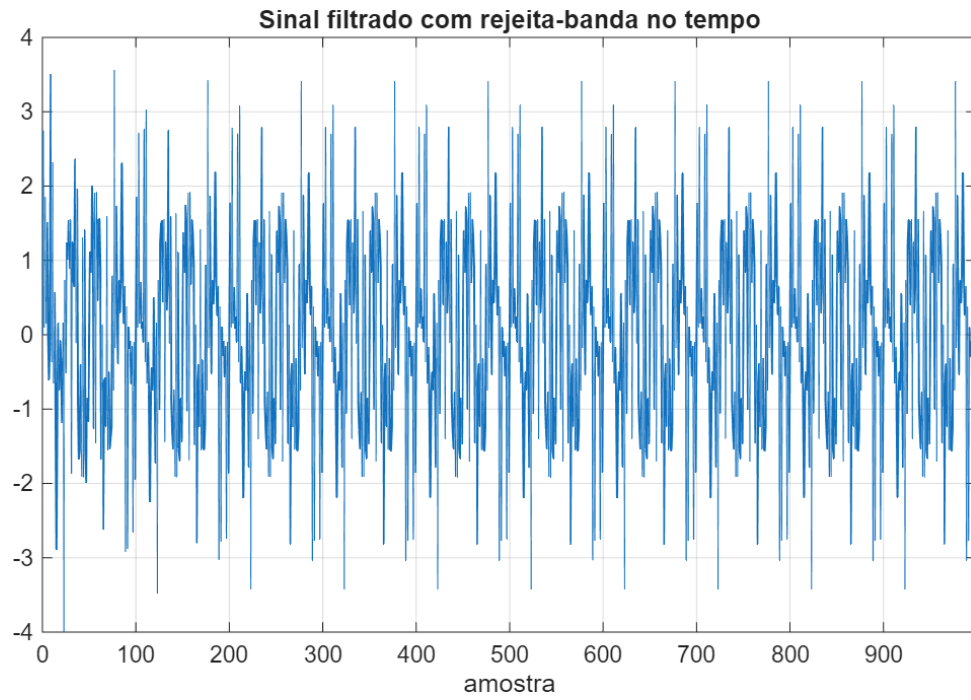


Figura 23: Sinal filtrado com o rejeita-banda (centrado em 250 Hz) no tempo.

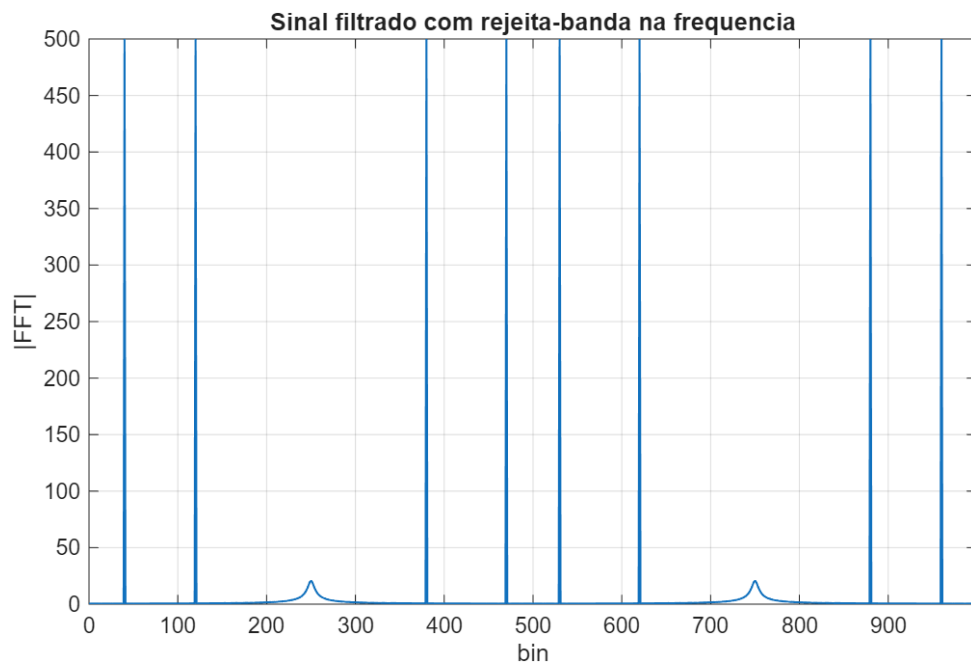


Figura 24: Sinal filtrado com o rejeita-banda na frequência: elimina a raia de 250 Hz e preserva as demais quatro componentes.

O passa-banda isola praticamente apenas a componente de 250 Hz (no tempo, o sinal filtrado tende a uma única senoide após o transitório inicial). Já o rejeita-banda elimina a raia de 250 Hz e preserva as demais quatro componentes, confirmando o comportamento esperado.

5 Processamento de Som

Enunciado: “Processamento de Som (em sala, se possível use fone de ouvido). A este sinal, adicionamos um ruído ‘branco’ apenas com as altas frequências, que não influencia as frequências do sinal do áudio. E, adicionalmente, um ruído com várias frequências próximas entre si, em um local não ocupado pelo espectro do áudio gravado. Após a inclusão dos ruídos, reverteu-se o espectro do sinal, como ocorre na operação de modulação (Equação 1) com o uso de uma portadora f_0 , com a máxima frequência possível para o tamanho do vetor gravado.”

$$x(t) \cos(2\pi f_0 t) \longleftrightarrow \frac{1}{2}[X(f - f_0) + X(f + f_0)] \quad (1)$$

(continuação do enunciado): “Agora, de posse dos conhecimentos adquiridos na disciplina, demodule o sinal e filtre os ruídos. Encontrando um sinal o mais parecido possível com o sinal original mostrado na Figura 1. Dica: utilize um filtro passa baixas Window-Sinc e um corta faixa Banda Estreita (neste pode ser preciso filtrar mais de uma vez)! Qual a frase demodulada e filtrada obtida? Gere um relatório explicando detalhadamente a tomada de decisão e os procedimentos usados para demodular o sinal e isola-lo do ruído. Colocando todos os códigos MATLAB e resultados obtidos.”

O arquivo `SinalRuidoModulado1s2026.mat` contém a variável `sinalModulado`, com 200.000 amostras adquiridas a $F_s = 40$ kHz (5 segundos). Sobre o sinal de voz original foram adicionados um ruído branco apenas nas altas frequências e um ruído de banda estreita (várias frequências próximas) em uma região não ocupada pela voz. Por fim, todo o espectro foi revertido pela operação de modulação da Equação 1, com portadora f_0 igual à máxima frequência possível (Nyquist).

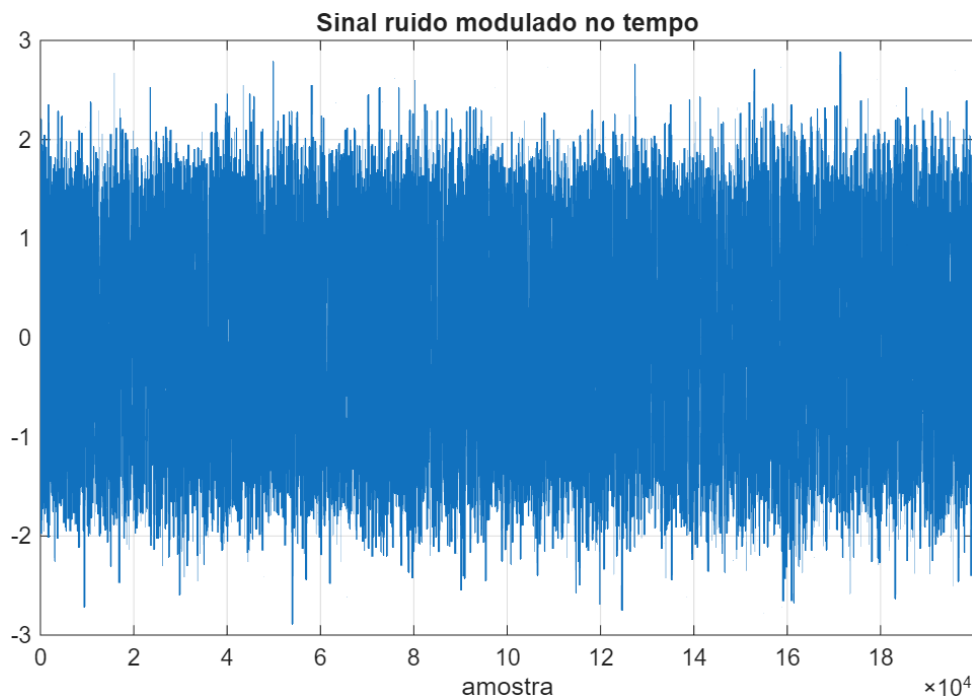


Figura 25: Sinal recebido (ruído modulado) no tempo.

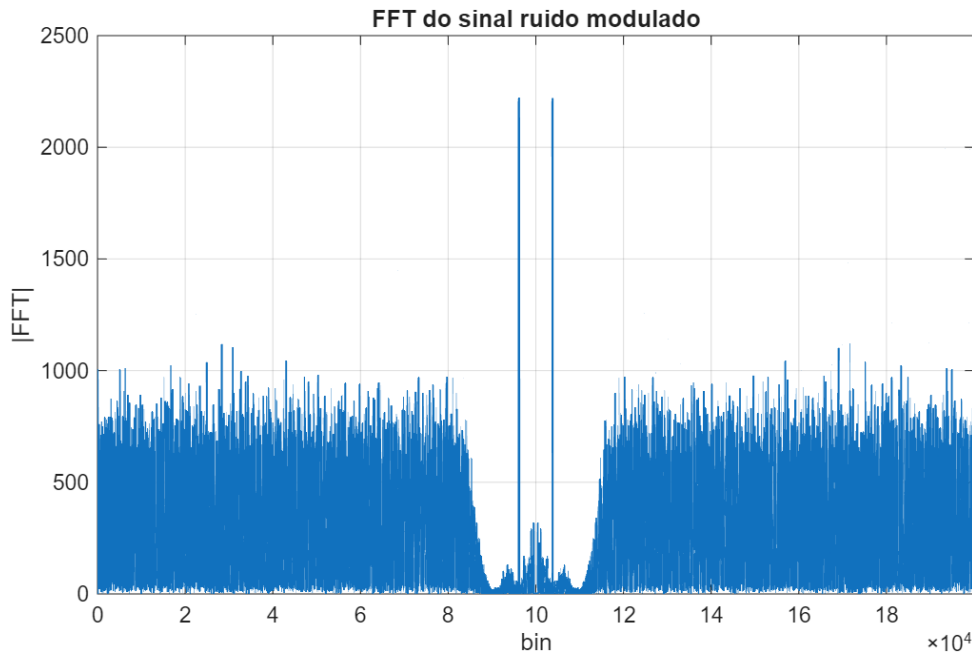


Figura 26: Sinal recebido (ruído modulado) na frequência. O espectro é dominado pelo ruído branco que ocupa quase toda a faixa.

Estratégia de recuperação

A recuperação foi dividida em três etapas: (i) demodulação espectral para trazer a voz de volta às baixas frequências; (ii) filtragem passa-baixas WindowSinc para eliminar o ruído branco de alta frequência; e (iii) rejeição de banda estreita, em cascata, para suprimir o tom interferente.

Demodulação espectral

Como a inversão do espectro foi feita com portadora na frequência de Nyquist ($f_0 = F_s/2 = 20$ kHz), a operação é a sua própria inversa. Basta multiplicar o sinal, amostra a amostra, por $\cos(\pi n) = (-1)^n$. Isso desloca todo o espectro de volta, recolocando a informação de áudio (que estava nas altas frequências) na faixa audível e enviando o ruído branco, que estava sobre o áudio, para a extremidade superior do espectro.

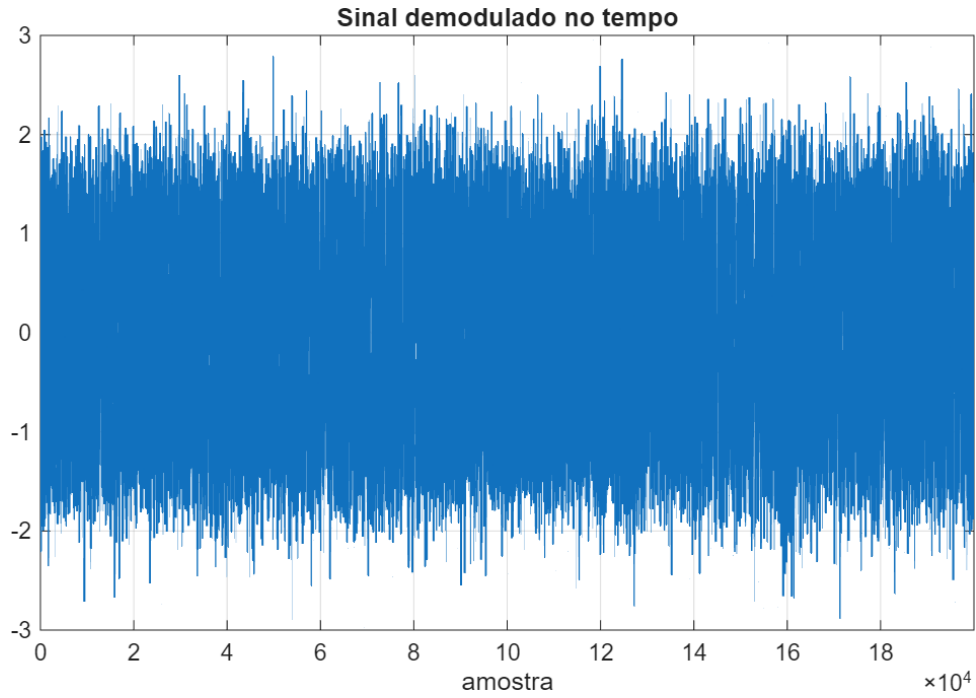


Figura 27: Sinal após a demodulação por $\cos(\pi n)$, no tempo.

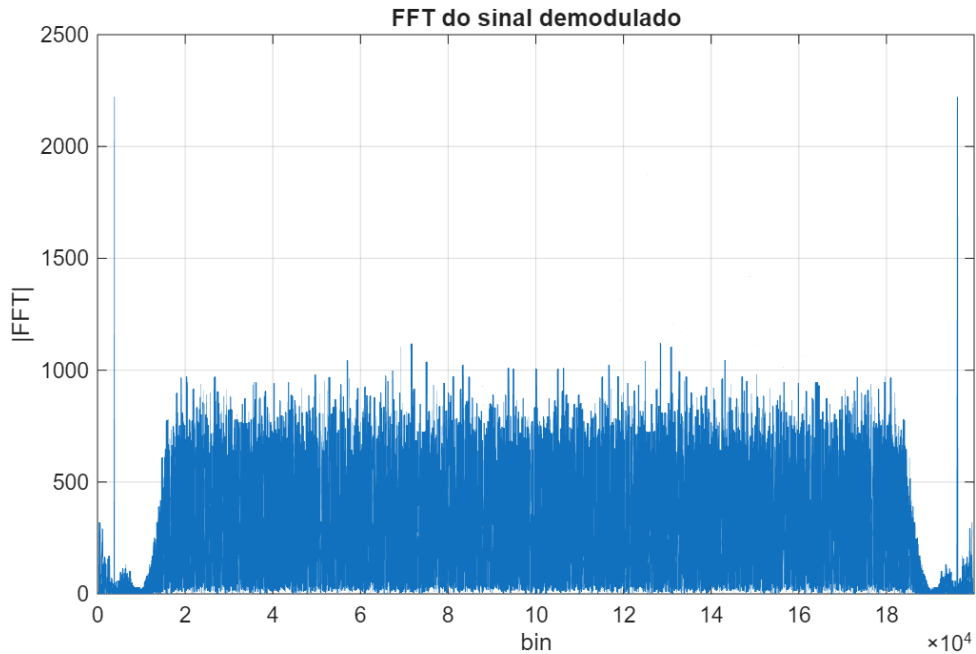


Figura 28: Sinal após a demodulação por $\cos(\pi n)$, na frequência. O espectro foi revertido: o conteúdo de voz reaparece em baixas frequências.

Filtragem passa-baixas (WindowSinc)

Observando o espectro demodulado, o ruído branco passa a ocupar as frequências acima de aproximadamente 2,9 kHz, enquanto a voz fica abaixo disso. Aplicou-se então um filtro passa-baixas WindowSinc (janela de Blackman, 100 coeficientes, frequência de corte normalizada $f_c = 0,075 \approx 3$ kHz). Por ter fase linear, a convolução preserva a integridade temporal da voz; o atraso de grupo foi compensado no recorte do sinal.

Listing 5: WindowSinc passa-baixas (janela de Blackman) usado no item 5.

```

1 function H = FWSPB(M, FC)
2 % FWSPB Filtro WindowSinc passa-baixas (FIR) com janela de Blackman.
3 % H = FWSPB(M, FC) retorna os M coeficientes de um nucleo sinc janelado,
4 % com frequencia de corte normalizada FC (0 < FC < 0.5). Ganho unitario
5 % em corrente continua (soma dos coeficientes = 1).
6     H = zeros(1, M);
7     m2 = M/2;
8     for i = 1:M
9         if (i - m2) == 0
10             H(i) = 2*pi*FC; % limite da sinc no centro
11         else
12             H(i) = sin(2*pi*FC*(i - m2)) / (i - m2);
13         end
14         w = 0.42 - 0.5*cos(2*pi*i/M) + 0.08*cos(4*pi*i/M); % janela de Blackman
15         H(i) = H(i) * w;
16     end
17     H = H / sum(H); % ganho unitario em DC
18 end

```

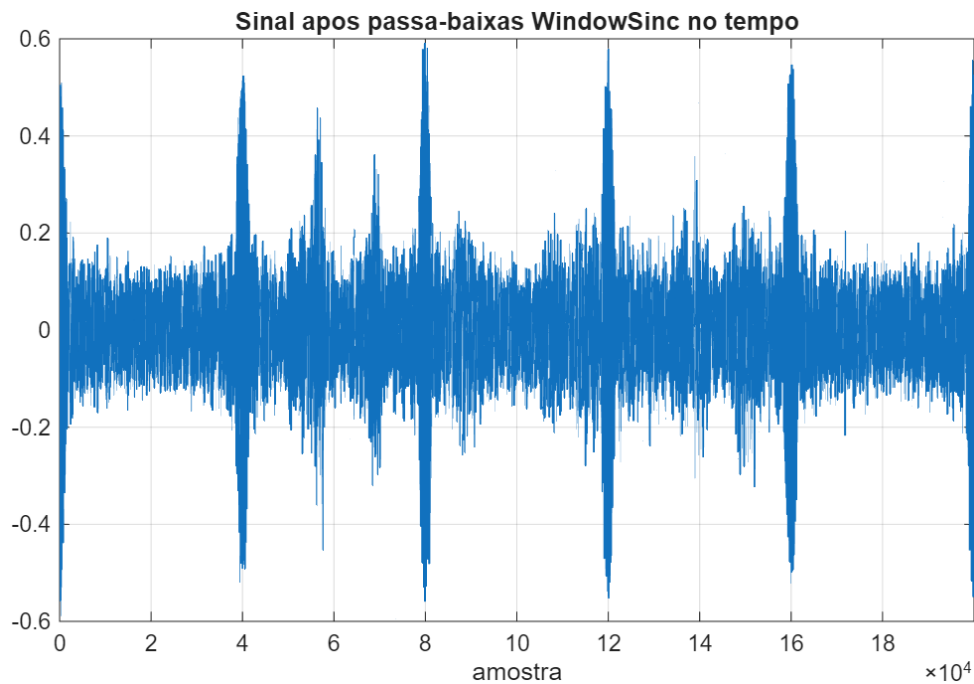


Figura 29: Sinal após o passa-baixas WindowSinc, no tempo.

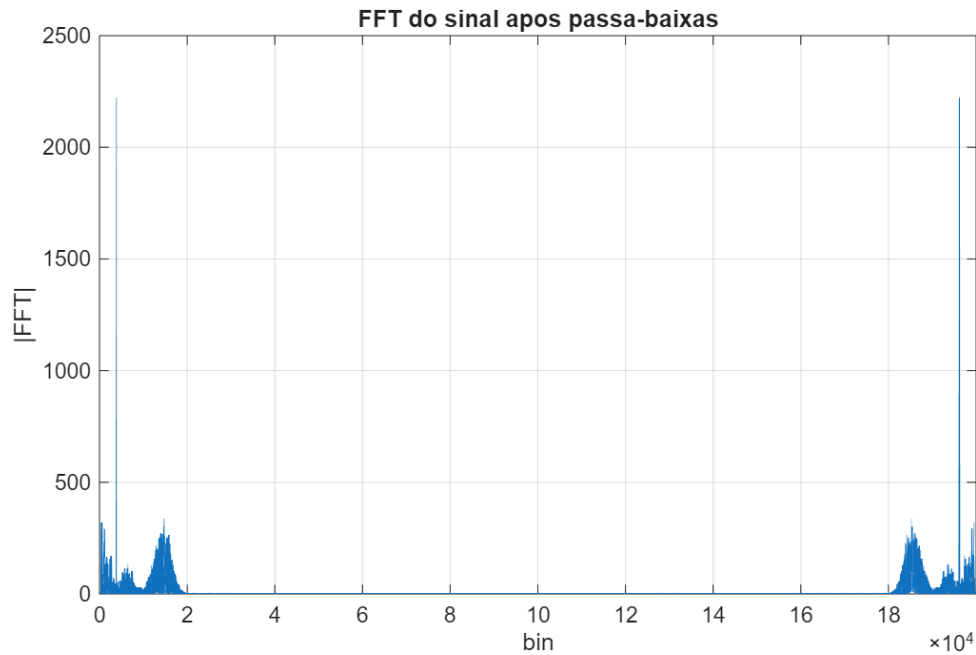


Figura 30: Sinal após o passa-baixas WindowSinc, na frequência. O ruído branco de alta frequência foi praticamente eliminado, restando a voz e um tom estreito.

Rejeição do tom de banda estreita

Mesmo após o passa-baixas, persiste um tom intenso e estreito em torno de 766 Hz (frequência normalizada $\approx 0,0192$), correspondente ao ruído de “várias frequências próximas”. Para suprimi-lo sem danificar a voz, usou-se um filtro rejeita-banda de banda muito estreita ($BW = 0,0008$) centrado nessa frequência, aplicado **três vezes em cascata**. Cada passagem aprofunda o entalhe e soma a sua atenuação, produzindo uma rejeição profunda do tom sem a necessidade de um filtro de ordem excessivamente elevada.

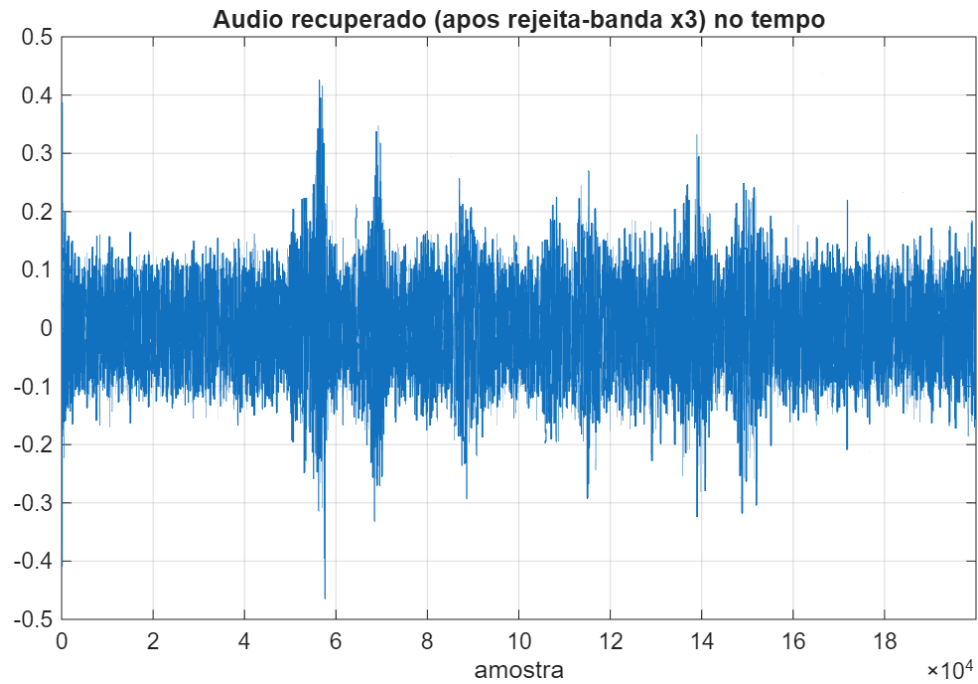


Figura 31: Áudio recuperado (após demodulação, passa-baixas e rejeita-banda em cascata) no tempo. Os surtos de amplitude correspondem às sílabas da frase falada.

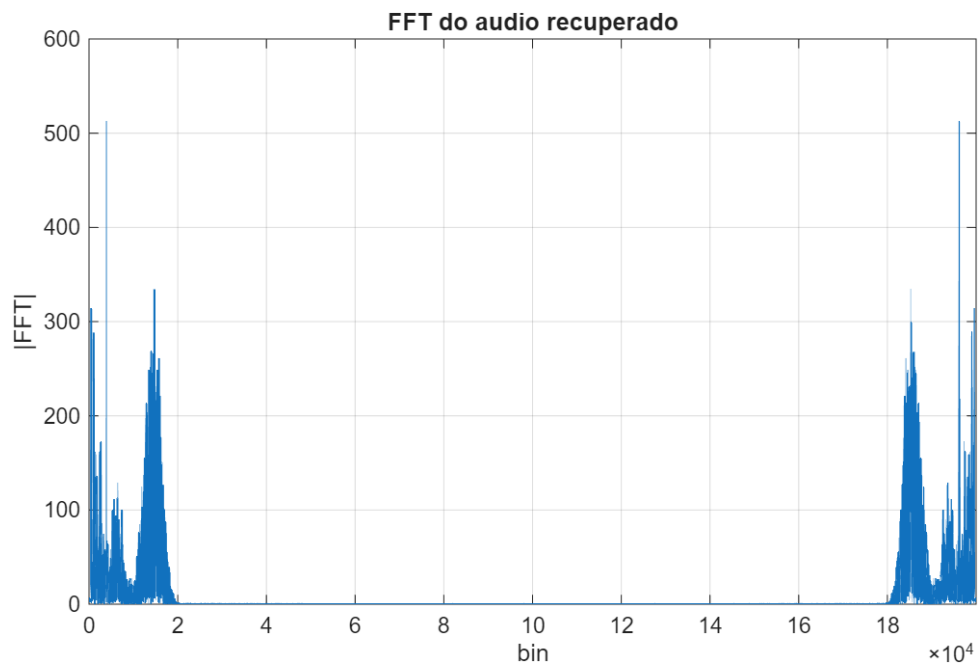


Figura 32: Áudio recuperado na frequência: a voz concentra-se nas baixas frequências e o tom interferente em 766 Hz foi praticamente eliminado.

O código completo do item 5 é apresentado a seguir.

Listing 6: Rotina de demodulação e filtragem do som (item 5).

```

1 %% =====
2 % Pratica 5 - Filtros IIR -- Item 5: Processamento de Som
3 % Demodulacao do espectro + filtragem dos ruidos do arquivo

```

```

4 % SinalRuidoModulado1s2026.mat
5 % Requer: FWSPB.m e FiltroRejeitaBanda.m (na mesma pasta)
6 % (em sala, se possivel use fone de ouvido)
7 % =====
8 clear; close all; clc;
9
10 % O arquivo .mat esta na pasta acima (raiz da "Pratica 5"); ajuste se preciso.
11 arquivo = fullfile('.', 'SinalRuidoModulado1s2026.mat');
12 load(arquivo); % carrega a variavel "sinalModulado"
13 sinal = reshape(sinalModulado, 1, []); % garante vetor linha
14 Fs = 40000; % frequencia de amostragem
15 N = length(sinal);
16 n = 0:N-1;
17
18 %% (a) Sinal recebido (ruído modulado)
19 figure('Name','Sinal modulado - tempo'); plot(sinal);
20 title('Sinal ruído modulado'); xlabel('amostra');
21 figure('Name','Sinal modulado - FFT'); plot(abs(fft(sinal)));
22 title('FFT do sinal ruído modulado'); xlabel('bin');
23
24 %% (b) Demodulação espectral: multiplicar por  $\cos(\pi \cdot n) = (-1)^n$ 
25 % Equivale a misturar com a portadora de Nyquist (20 kHz), revertendo o
26 % espectro: o áudio volta para as baixas frequências.
27 demodulado = sinal .* cos(pi*n);
28 figure('Name','Sinal demodulado - tempo'); plot(demodulado);
29 title('Sinal demodulado'); xlabel('amostra');
30 figure('Name','Sinal demodulado - FFT'); plot(abs(fft(demodulado)));
31 title('FFT do sinal demodulado'); xlabel('bin');
32
33 %% (c) Passa-baixas WindowSinc: remove o ruído branco de alta frequência
34 fc = 0.075; % ~3 kHz (borda observada onde começa o ruído branco)
35 M = 100;
36 h = FWSPB(M, fc);
37 lp = conv(demodulado, h);
38 lp = lp(M/2+1 : M/2+N); % alinha (remove atraso de grupo) e mantém N amostras
39 figure('Name','Apos passa-baixas - tempo'); plot(lp);
40 title('Sinal apos passa-baixas WindowSinc'); xlabel('amostra');
41 figure('Name','Apos passa-baixas - FFT'); plot(abs(fft(lp)));
42 title('FFT apos passa-baixas'); xlabel('bin');
43
44 %% (d) Rejeita-banda estreito no tom interferente (~766 Hz), em cascata 3x
45 fnotch = 766/Fs; % frequência normalizada do tom (~0.01915)
46 bw = 0.0008; % banda muito estreita
47 rb = lp;
48 for k = 1:3 % cascata: soma as atenuações de cada estágio
49     rb = FiltroRejeitaBanda(bw, fnotch, rb);
50 end
51 figure('Name','Áudio recuperado - tempo'); plot(rb);
52 title('Áudio recuperado (apos rejeita-banda x3)'); xlabel('amostra');
53 figure('Name','Áudio recuperado - FFT'); plot(abs(fft(rb)));
54 title('FFT do áudio recuperado'); xlabel('bin');
55
56 %% (e) Reproduz e salva o resultado
57 sinal_recuperado = rb / max(abs(rb)) * 0.95;
58 sound(sinal_recuperado, Fs); % ouça a frase recuperada
59 audiowrite('sinal_recuperado.wav', sinal_recuperado, Fs);
60 save('sinal_recuperado.mat', 'sinal_recuperado');
61 disp('Item 5 concluído: ouça o áudio e anote a frase recuperada.');
```


Frase obtida

Após a demodulação e a filtragem, o sinal recuperado foi salvo em `sinal_recuperado.mat` e `sinal_recuperado.wav`. A frase demodulada e filtrada obtida foi:

“Será que o Hexa vem ou vamos ensacar a viola?”

6 Conclusão

A prática consolidou os conceitos fundamentais dos filtros IIR. A implementação dos filtros de pólo simples permitiu observar diretamente a relação entre a constante r , a frequência de corte e o decaimento da resposta ao impulso, bem como o comportamento suave das curvas de ganho e fase de um sistema de primeira ordem. Os filtros de banda estreita, de segunda ordem, evidenciaram o papel do parâmetro BW no posicionamento dos pólos e na seleção (ou rejeição) de uma faixa estreita do espectro.

A etapa de processamento de som foi a mais completa: combinando demodulação espectral, um filtro passa-baixas WindowSinc de fase linear e um rejeita-banda estreito aplicado em cascata, foi possível reverter a inversão do espectro, remover o ruído branco de alta frequência e suprimir o tom interferente, recuperando a mensagem de voz original de forma inteligível. O exercício reforçou a importância de identificar corretamente, no domínio da frequência, as faixas de interesse e de projetar o filtro adequado a cada etapa do processamento.